

```

# Evaluate the game's win/lose/tie conditions
❷ if playerChoice == computerChoice:
    self.resultsField.setValue("It's a tie!")
    self.tieSound.play()

elif playerChoice == ROCK and computerChoice == SCISSORS:
    self.resultsField.setValue("Rock breaks Scissors. You win!")
    self.playerScore = self.playerScore + 1
    self.winnerSound.play()

--- snip ---

# Show the player's and computer's scores
self.playerScoreCounter.setValue(
    'Score: ' + str(self.playerScore))
self.computerScoreCounter.setValue(
    'Score: ' + str(self.computerScore))

❸ def handleInputs(self, eventsList, keyPressedList):
    for event in eventsList:
        if self.restartButton.handleEvent(event):
            self.goToScene(SCENE_PLAY)

# No need to include update method,
# defaults to inherited one which does nothing

❹ def draw(self):
    self.window.fill(OTHER_GRAY)
    self.youComputerField.draw()
    self.resultsField.draw()
    self.rpsCollectionPlayer.draw()
    self.rpsCollectionComputer.draw()
    self.playerScoreCounter.draw()
    self.computerScoreCounter.draw()
    self.restartButton.draw()

```

Listing 15-5: The Results scene in Rock, Paper, Scissors

Here, I've snipped some of the game evaluation logic. The `enter()` method ❷ is the most important method in this class. When the player makes a choice in the previous Play scene, the program transitions to this Results scene. First, we extract the player's and computer's choices that were passed in from the Play scene as a dictionary, which looks like this: